

Project #1

Surname, Name	Student Number
Doğa ÖZBEK	21602981

Q1	Q2	Q3	Q4	Q5

Quote	"The work I here deliver within is my own and self-output."
Signature	

A **Model F-16C** aircraft has the following **geometric, mass and aerodynamic model parameters** in very low subsonic flight at very low altitudes.



Figure for Q1: The F-16C Aircraft.

```

% -----
% A/C: mass parameters
% -----
m          = 9298.643580;           % [kg]
Jxx        = 12874.84725;           % [kg/m^2]
Jyy        = 75673.62306;           % [kg/m^2]
Jzz        = 85552.11265;           % [kg/m^2]
Jxz        = 1331.413230;           % [kg/m^2]
Sref       = 27.87091;              % [m^2]
cbar       = 3.45034;               % [m]
b          = 9.14400;               % [m]
xcg        = 0.86258;               % [m]

% -----
% A/C: aerodynamic parameters
% -----

```

THETA1 = 0.03400;	THETA16 = 0.07400;	THETA31 = -8.26700;
THETA2 = -0.00500;	THETA17 = 4.45800;	THETA32 = -0.56300;
THETA3 = 20.77000;	THETA18 = 29.90000;	THETA33 = -5.51300;
THETA4 = 0.17700;	THETA19 = 0.41200;	THETA34 = 9.79300;
THETA5 = 1.28500;	THETA20 = -5.53800;	THETA35 = -1.05700;
THETA6 = -19.9700;	THETA21 = -2.47700;	THETA36 = -2.01800;
THETA7 = 0.75600;	THETA22 = -1.10100;	THETA37 = 1.89700;
THETA8 = 5.88700;	THETA23 = 1.90600;	THETA38 = -0.09400;
THETA9 = 55.59000;	THETA24 = -0.07100;	THETA39 = 0.23400;
THETA10 = -5.15500;	THETA25 = -0.44500;	THETA40 = 0.05600;
THETA11 = -1.14600;	THETA26 = 0.05800;	THETA41 = -0.41800;
THETA12 = -0.18800;	THETA27 = -0.14300;	THETA42 = -0.03400;
THETA13 = 0.87600;	THETA28 = 0.02300;	THETA43 = -0.08500;
THETA14 = 0.06000;	THETA29 = -0.02400;	THETA44 = 0.37200;
THETA15 = 0.16400;	THETA30 = -0.28800;	THETA45 = -0.72500;

The non-linear functions to build the aerodynamic coefficients, for this A/C, are also presented below.

```
% -----
% A/C: aerodynamic force and moment coefficients
% -----
CD = THETA1 + THETA2*alpha + THETA3*alpha*q*cbar/2/VT + THETA4*alpha*de +
THETA5*alpha^2 + THETA6*alpha^2*q*cbar/2/VT + THETA7*alpha^2*de + THETA8*alpha^3 +
THETA9*alpha^3*q*cbar/2/VT + THETA10*alpha^4;
CY = THETA11*beta + THETA12 *p*b/2/VT + THETA13*r*b/2/VT + THETA14*da + THETA15*dr;
CL = THETA16 + THETA17*alpha + THETA18*q*cbar/2/VT + THETA19*de +
THETA20*alpha*q*cbar/2/VT + THETA21*alpha^2 + THETA22*alpha^3 + THETA23*alpha^4;
% -----
Cl = THETA24*beta + THETA25 *p*b/2/VT + THETA26*r*b/2/VT + THETA27*da + THETA28*dr;
Cm = THETA29 + THETA30*alpha + THETA31*q*cbar/2/VT + THETA32*de +
THETA33*alpha*q*cbar/2/VT + THETA34*alpha^2*q*cbar/2/VT + THETA35*alpha^2*de +
THETA36*alpha^3*q*cbar/2/VT + THETA37*alpha^3*de + THETA38*alpha^4;
Cn = THETA39*beta + THETA40 *p*b/2/VT + THETA41*r*b/2/VT + THETA42*da + THETA43*dr +
THETA44*beta^2 + THETA45*beta^3;
% -----
```

The thrust force and moment for the A/C are constructed as shown in the following lines.

```
% -----
% A/C: thrust force and moment components
% -----
% Qd      : Dynamic pressure.
% Cthr     : Thrust force coefficient.
% thetaEngine: Engine installation angle.
% C_bT     = R2(thetaEngine);
% -----
thetaEngine = 0.05236;           % 3 degrees           % [rad]
% -----
Thrust      = C_bT * [Qd * Sref * Cthr;0;0];
% -----
Ftx         = Thrust(1,1);
Fty         = Thrust(2,1);
Ftz         = Thrust(3,1);
% -----
Mtx         = 0;
Mty         = 0;
Mtz         = 0;
% -----
```

The flight and initial conditions for the A/C are listed below.

```
% -----
% A/C: flight and initial conditions
% -----
Mach        = 0.15;               % [-]
h           = 100;                % z = -h;           % [m]
% -----
x           = 0;                  % [m]
y           = 0;                  % [m]
% -----
chi         = 0/180*pi;           % [rad]
gamma       = 0/180*pi;           % [rad]
mu          = 0/180*pi;           % [rad]
% -----
```

```
% -----
alpha    = alpha_Trim;           %      [rad]
beta     = 0/180*pi;            %      [rad]
% -----
p        = 0/180*pi;            %      [rad/s]
q        = 0/180*pi;            %      [rad/s]
r        = 0/180*pi;            %      [rad/s]

% -----
% A/C: control commands
% -----
de       = de_Trim;              %      [rad]
Cthr     = Cthr_Trim;            %      [-]
da       = 0/180*pi;            %      [rad]
dr       = 0/180*pi;            %      [rad]
% -----
```

QUESTIONS

1. Calculate the **trim values at wings-level steady flight**. (25 pts.)

```
% -----
% A/C: trim conditions
% -----
alpha_Trim = ?                   %      [rad]
de_Trim    = ?                   %      [rad]
Cthr_Trim  = ?                   %      [rad]
% -----
```

Here, you can use any tool to solve “coupled nonlinear equations”. Or, you can write your own code by investigating certain numerical recipes for solution of nonlinear equation sets, root finding, and etc.

2. Calculate the **longitudinal and lateral-directional static stability** derivatives. (15 pts.)

Is the A/C statically stable (longitudinal and lateral-directional)? Shortly explain.

3. Write a **6DoF simulation code**. (25 pts.)

Simulate (90 seconds) and **plot** the following variables (as a function of time).

Is the A/C in wings-level trimmed steady flight? Shortly explain.

```
% -----
% A/C: plot
% -----
figure(1), plot(t,Mach(t))      & plot(t,alpha(t))      & plot(t,beta(t))
figure(2), plot(t,p(t))         & plot(t,q(t))         & plot(t,r(t))
figure(3), plot(t,phi(t))       & plot(t,theta(t))     & plot(t,psi(t))
% -----
```

4. Use your **6DoF simulation** code and comment on the **longitudinal and lateral-directional dynamic stability**. (15 pts.)

To investigate the **longitudinal dynamic stability**, keep the initial conditions same and apply following control commands.

```
% -----
% A/C: control commands
% -----
cont_on_off = (t >= 2 & t < 4) - (t >= 4 & t <= 6);
delay       = dsp.Delay(6/dt);
trig_lon    = 1;
trig_lat    = 0;
% -----
de          = de_Trim + trig_lon * de_Trim/10 * cont_on_off;
Cthr       = Cthr_Trim + trig_lon * Cthr_Trim/10 * delay(cont_on_off);
da         = 0/180*pi - trig_lat * 1/180*pi * cont_on_off;
dr         = 0/180*pi - trig_lat * 1/180*pi * delay(cont_on_off);
% -----
```

To investigate the **lateral-directional dynamic stability**, keep the initial conditions same and apply following control commands.

```
% -----
% A/C: control commands
% -----
cont_on_off = (t >= 2 & t < 4) - (t >= 4 & t <= 6);
delay       = dsp.Delay(6/dt);
trig_lon    = 0;
trig_lat    = 1;
% -----
de          = de_Trim + trig_lon * de_Trim/10 * cont_on_off;
Cthr       = Cthr_Trim + trig_lon * Cthr_Trim/10 * delay(cont_on_off);
da         = 0/180*pi - trig_lat * 1/180*pi * cont_on_off;
dr         = 0/180*pi - trig_lat * 1/180*pi * delay(cont_on_off);
% -----
```

Simulate (90 seconds) and **plot** the following variables (as a function of time).

Is the A/C dynamically stable (longitudinal and lateral-directional)? Shortly explain.

```
% -----
% A/C: plot
% -----
figure(1), plot(t,Mach(t)) & plot(t,alpha(t)) & plot(t,beta(t))
figure(2), plot(t,p(t)) & plot(t,q(t)) & plot(t,r(t))
figure(3), plot(t,phi(t)) & plot(t,theta(t)) & plot(t,psi(t))
% -----
```

5. Construct the **6DoF small perturbation equations** and comment on the **longitudinal and lateral-directional dynamic stability**. (25 pts.)

Here, you can use any tool to linearize “coupled nonlinear equations”. Or, you can write your own code by investigating certain numerical recipes for linearization of nonlinear equation sets.

Is the A/C dynamically stable (longitudinal and lateral-directional)? Shortly explain.

Hint: Calculate the eigen-values of the system state matrix $\hat{A}$, and classify the rigid-body modes of the A/C.

Notes

- 1) Please rely on the notations, derivations and information given in class and extra materials.
- 2) You can handle your Project as being hand-written (and scanned) or printed. However, in any case, please present your work in a very neat manner.
- 3) Present your answers with graphic(s) and plot(s), where necessary.

You are supposed to handle all of your plot(s) printed on A4 size papers and printed in color.

- 4) Submit your code(s) (**Simulink® is not allowed**), to TA of ME 570.

The codes, which are either the same or looking closely similar, will not be counted and graded.

- 5) Please print the first page and sign below the pledge.
This printed (and signed) page should be the first and/or cover page of your work.

```

clear all
close all
clc
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%PART A==> Q1
%PART B==> Q2
%PART C==> Q3
%PART D==> Q4
%PART E==> Q5

global m Jxx Jyy Jzz Jxz Sref cbar b xcg THETA1 THETA2 THETA3...
      THETA4 THETA5 THETA6 THETA7 THETA8 THETA9 THETA10 THETA11
      THETA12...
      THETA13 THETA14 THETA15 THETA16 THETA17 THETA18 THETA19 THETA20
      THETA21...
      THETA22 THETA23 THETA24 THETA25 THETA26 THETA27 THETA28 THETA29
      THETA30...
      THETA31 THETA32 THETA33 THETA34 THETA35 THETA36 THETA37 THETA38
      THETA39 THETA40...
      THETA41 THETA42 THETA43 THETA44 THETA45 thetaEngine Mtx Mty Mtz
Mach h x y chi gamma mu ...
      beta p q r da dr data_atmosphere T P ro a nu VT g0 RE g R Qd phi
psi z
%
-----
% A/C: mass parameters
%
-----
m = 9298.643580; % [kg]
Jxx = 12874.84725; % [kg/m^2]
Jyy = 75673.62306; % [kg/m^2]
Jzz = 85552.11265; % [kg/m^2]
Jxz = 1331.413230; % [kg/m^2]
Sref = 27.87091; % [m^2]
cbar = 3.45034; % [m]
b = 9.14400; % [m]
xcg = 0.86258; % [m]
%
-----
% A/C: aerodynamic parameters
%
-----
THETA1 = 0.03400;
THETA2 = -0.00500;
THETA3 = 20.77000;
THETA4 = 0.17700;
THETA5 = 1.28500;
THETA6 = -19.9700;
THETA7 = 0.75600;
THETA8 = 5.88700;

```

```

THETA9 = 55.59000;
THETA10 = -5.15500;
THETA11 = -1.14600;
THETA12 = -0.18800;
THETA13 = 0.87600;
THETA14 = 0.06000;
THETA15 = 0.16400;
THETA16 = 0.07400;
THETA17 = 4.45800;
THETA18 = 29.90000;
THETA19 = 0.41200;
THETA20 = -5.53800;
THETA21 = -2.47700;
THETA22 = -1.10100;
THETA23 = 1.90600;
THETA24 = -0.07100;
THETA25 = -0.44500;
THETA26 = 0.05800;
THETA27 = -0.14300;
THETA28 = 0.02300;
THETA29 = -0.02400;
THETA30 = -0.28800;
THETA31 = -8.26700;
THETA32 = -0.56300;
THETA33 = -5.51300;
THETA34 = 9.79300;
THETA35 = -1.05700;
THETA36 = -2.01800;
THETA37 = 1.89700;
THETA38 = -0.09400;
THETA39 = 0.23400;
THETA40 = 0.05600;
THETA41 = -0.41800;
THETA42 = -0.03400;
THETA43 = -0.08500;
THETA44 = 0.37200;
THETA45 = -0.72500;

```

```
%
```

```
-----
% A/C: aerodynamic force and moment coefficients
```

```
%
```

```
-----
% CD = THETA1 + THETA2*alpha + THETA3*alpha*q*cbar/2/VT +
  THETA4*alpha*de + ...
% THETA5*alpha^2 + THETA6*alpha^2*q*cbar/2/VT + THETA7*alpha^2*de +
  THETA8*alpha^3 + ...
% THETA9*alpha^3*q*cbar/2/VT + THETA10*alpha^4;
% CY = THETA11*beta + THETA12 *p*b/2/VT + THETA13*r*b/2/VT +
  THETA14*da + THETA15*dr;
% CL = THETA16 + THETA17*alpha + THETA18*q*cbar/2/VT + THETA19*de + ...
% THETA20*alpha*q*cbar/2/VT + THETA21*alpha^2 + THETA22*alpha^3 +
  THETA23*alpha^4;

```

```

%
-----
% Cl = THETA24*beta + THETA25 *p*b/2/VT + THETA26*r*b/2/VT +
  THETA27*da + THETA28*dr;
% Cm = THETA29 + THETA30*alpha + THETA31*q*cbar/2/VT + THETA32*de +...
% THETA33*alpha*q*cbar/2/VT + THETA34*alpha^2*q*cbar/2/VT +
  THETA35*alpha^2*de +...
% THETA36*alpha^3*q*cbar/2/VT + THETA37*alpha^3*de + THETA38*alpha^4;
% Cn = THETA39*beta + THETA40 *p*b/2/VT + THETA41*r*b/2/VT +
  THETA42*da + THETA43*dr +...
% THETA44*beta^2 + THETA45*beta^3;
%
-----

%
-----
% A/C: thrust force and moment components
%
-----
% Qd : Dynamic pressure.
% Cthr : Thrust force coefficient.
% thetaEngine: Engine installation angle.
% C_bT = R2(thetaEngine);
%
-----
thetaEngine = 0.05236; % 3 degrees % [rad]
%
-----
%Thrust = C_bT * [Qd * Sref * Cthr;0;0];
%
-----
%Ftx = Thrust(1,1);
%Fty = Thrust(2,1);
%Ftz = Thrust(3,1);
%
-----
Mtx = 0;
Mty = 0;
Mtz = 0;
%
-----

%
-----
% A/C: flight and initial conditions
%
-----
Mach = 0.15; % [-]
h = 100;
z = -h; % [m]
%
-----
x = 0; % [m]
y = 0; % [m]

```

```

%
-----
chi = 0/180*pi; % [rad]
gamma = 0/180*pi; % [rad]
mu = 0/180*pi; % [rad]
phi=0; %[rad] since this is wing steady flight
psi=0; %[rad] since this is wing steady flight
%
-----

%
-----
%alpha = alpha_Trim; % [rad]
alpha=0; %initializing
beta = 0/180*pi; % [rad]
theta=gamma+alpha;
%
-----

p = 0/180*pi; % [rad/s]
q = 0/180*pi; % [rad/s]
r = 0/180*pi; % [rad/s]
%
-----

% A/C: control commands
%
-----

%de = de_Trim; % [rad]
de=0; %initializing
%Cthr = Cthr_Trim; % [-]
Cthr=0;
da = 0/180*pi; % [rad]
dr = 0/180*pi; % [rad]
%
-----

%Calculation of the necessary data

%By using f_atmos_US76 function we have air properties;

data_athmosphere=f_atmos_US76(h);
T=data_athmosphere(1); %K
P=data_athmosphere(2); %N/m^2
ro=data_athmosphere(3); %kg/m^3
a=data_athmosphere(4); %m/s
nu=data_athmosphere(5); %kg/ms

VT=Mach*a; % A/C speed in m/s
R = 287.05287; % value from ESDU 77022 [Nm/kgK]
g0 = 9.80665; % [m/sec^2]
RE = 6356766; % radius of the Earth [m]
g=g0*(RE/(RE+h))^2; % [m/sec^2]
Qd=0.5*ro*VT^2;

%PART A)

```

```

coefficients=calcoef(alpha,da,de,dr,p,q,r,VT,beta);
[total_force,total_moment,weight]=calcFM(coefficients,alpha,
    Cthr,theta,phi,VT);
opts = optimoptions(@fsolve,'Display','off');
X_0 = [alpha de Cthr]';
solution = fsolve(@wing_level,X_0, opts);

fprintf("-----Comments on the part A-----\n");

fprintf("MATLAB optimization toolbox is used to find
    alpha_trim,de_trim and Cthr_trim \n");
fprintf("by considering trim conditions.\n");

alpha_trim=solution(1)
de_trim=solution(2)
Cthr_trim=solution(3)

VT=Mach*a;

%PART B)
alpha=alpha_trim;
de=de_trim;
Cthr=Cthr_trim;

%For Longitudinal direction

dCm_dAlpha = THETA30 + THETA33*q*cbar/2/VT + 2*THETA34*alpha*q*cbar/2/
VT+...
            2*THETA35*alpha*de+3*THETA36*alpha^2*q*cbar/2/VT ...
            + 3*THETA37*alpha^2*de + 4*THETA38*alpha^3;
dCl_dBeta = THETA24;
dCn_dBeta = THETA39 + 2*THETA44*beta + 3*THETA45*beta^2;

fprintf("-----Comments on the part B-----\n");

if dCm_dAlpha < 0
    fprintf("dCm/dAlpha = %f\n",dCm_dAlpha);
    fprintf("Since Longitudinal Static Stability Derivative is
        negative, \n");
    fprintf("therefore, the A/C is stable in longitudinal direction
        plane.\n");
else
    fprintf("dCm/dAlpha = %f\n",dCm_dAlpha);
    fprintf("Since Longitudinal Static Stability Derivative is
        positive,\n");
    fprintf("therefore the A/C is not stable in longitudinal direction
        plane.\n");
end

%For Lateral direction

if dCn_dBeta > 0 && dCl_dBeta < 0

```

```

        fprintf("dCl/dBeta = %f\n",dCl_dBeta);
        fprintf("dCn/dBeta = %f\n",dCn_dBeta);
        fprintf("Since lateral Static Stability Derivative is negative and
directional\n");
        fprintf("static stability is positive. In this case we can
conclude that\n");
        fprintf("The A/C is stable in directional plane.\n");
else
        fprintf("dCn/dBeta = %f\n",dCn_dBeta);
        fprintf("Since Directional Static Stability Derivative is\n");
        fprintf("negative. Therefore, The A/C is not stable in directional
plane.\n");
end

VT=Mach*a;
%PART C)

%necessary parameters for ODE45 solver

dummy_vector=R_2(-alpha)*R_3(beta)*[VT;0;0];
u=dummy_vector(1);
v=dummy_vector(2);
w=dummy_vector(3);
theta=alpha+gamma;
initial_vector=[u;v;w;p;q;r;phi;theta;psi];
t_span=0:0.1:90;
[t,solution2] = ode45(@(t,Z)odefcn(t,Z, alpha, de,
Cthr),t_span,initial_vector);

%data export
u_exp=solution2(:,1);
v_exp=solution2(:,2);
w_exp=solution2(:,3);
p_exp=solution2(:,4);
q_exp=solution2(:,5);
r_exp=solution2(:,6);
phi_exp=solution2(:,7);
theta_exp=solution2(:,8);
psi_exp=solution2(:,9);

%processed data
V_exp=sqrt(u_exp.^2+v_exp.^2+w_exp.^2);
mach_exp=V_exp/a;
alpha_exp=atan2(w_exp,u_exp);
beta_exp=asin(v_exp./V_exp);

% comment part for part C
fprintf("-----Comments on the part C-----
\n");
fprintf("From the plots that we have, It can be observed that all of
the variables stayed\n");
fprintf("almost constant and during the simulation time period p,q,r
was stayed\n");

```

```

fprintf("at zero. As a result, we can conclude that this means the A/
C\n");
fprintf("is in wings level trimmed steady flight.\n");

VT=Mach*a;

```

PART D %%%%%%%%%

```

----- A/C: control commands
-----

cont_on_off = (t >= 2 & t < 4) - (t >= 4 & t <= 6);
dt=0.1;
delay = dsp.Delay(6/dt);
trig_lon = 1;
trig_lat = 0;
%
-----
deD = de_trim + trig_lon * de_trim/10 * cont_on_off;
CthrD = Cthr_trim + trig_lon * Cthr_trim/10 * delay(cont_on_off);
daD = 0/180*pi - trig_lat * 1/180*pi * cont_on_off;
drD = 0/180*pi - trig_lat * 1/180*pi * delay(cont_on_off);
%
-----

%solution3 includes the solution for longitudinal dynamic equations

[t,solution3] = ode45(@(t,Z)odefcn2(t,dt,Z, alpha, deD, CthrD,drD,
daD),t_span,initial_vector);

%data export
u_exp2=solution3(:,1);
v_exp2=solution3(:,2);
w_exp2=solution3(:,3);
p_exp2=solution3(:,4);
q_exp2=solution3(:,5);
r_exp2=solution3(:,6);
phi_exp2=solution3(:,7);
theta_exp2=solution3(:,8);
psi_exp2=solution3(:,9);

%processed data
V_exp2=sqrt(u_exp2.^2+v_exp2.^2+w_exp2.^2);
mach_exp2=V_exp2/a;
alpha_exp2=atan2(w_exp2,u_exp2);
beta_exp2=asin(v_exp2./V_exp2);

dCm_dAlpha_Dyn_Lo = THETA30 + THETA33*q*cbar/2./V_exp2 + ...
2*THETA34.*alpha_exp2.*q*cbar/2./V_exp2 +
2*THETA35.*alpha_exp2.*de ...
+ 3*THETA36.*alpha_exp2.^2*q*cbar/2./V_exp2 +
3*THETA37.*alpha_exp2.^2.*de ...
+ 4*THETA38.*alpha_exp2.^3;

```

```

%For Stability check

boolean_vector=(dCm_dAlpha_Dyn_Lo>0);
summer=sum(boolean_vector);

fprintf("-----Comments on the part D-----\n");

if summer==0
    disp('A/C is stable in longitudinal direction since the
    longitudinal stability derivative contains no positive value. ')
else
    disp('A/C is unstable in longitudinal direction since the
    longitudinal stability derivative contains positive value. ')
end

%for lateral dynamic calculations and stability check
%
-----
% A/C: control commands
%
-----
cont_on_off = (t >= 2 & t < 4) - (t >= 4 & t <= 6);
dt=0.1;
delay = dsp.Delay(6/dt);
trig_lon = 0;
trig_lat = 1;
%
-----
deD = de_trim + trig_lon * de_trim/10 * cont_on_off;
CthrD = Cthr_trim + trig_lon * Cthr_trim/10 * delay(cont_on_off);
daD = 0/180*pi - trig_lat * 1/180*pi * cont_on_off;
drD = 0/180*pi - trig_lat * 1/180*pi * delay(cont_on_off);
%
-----

%solution4 includes the solution for longitudinal dynamic equations

VT=Mach*a;
[t,solution4] = ode45(@(t,Z)odefcn2(t,dt,Z, alpha, deD, CthrD,drD,
    daD),t_span,initial_vector);

%data export
u_exp3=solution4(:,1);
v_exp3=solution4(:,2);
w_exp3=solution4(:,3);
p_exp3=solution4(:,4);
q_exp3=solution4(:,5);
r_exp3=solution4(:,6);
phi_exp3=solution4(:,7);
theta_exp3=solution4(:,8);
psi_exp3=solution4(:,9);

```

```

%processed data
V_exp3=sqrt(u_exp3.^2+v_exp3.^2+w_exp3.^2);
mach_exp3=V_exp3/a;
alpha_exp3=atan2(w_exp3,u_exp3);
beta_exp3=asin(v_exp3./V_exp3);

dCn_dBeta=THETA39+2*THETA44.*beta_exp3 + 3*THETA45.*beta_exp3.^2;
dCl_dBeta=THETA24;

boolean_vector2=(dCn_dBeta<0);
summer2=sum(boolean_vector2);

if summer2==0
    if dCl_dBeta<0
        disp('A/C is unstable in the lateral direction since dCn/dBeta
        does not have negative value and dCl/dBeta is negative.')
    end
else
    disp('A/C is unstable in the lateral diretion since the lateral
    stability derivative contains negative value. ')
end

%%PART E)

dt=thetaEngine;

%longitudinal motion

VT=Mach*a;
i0_long=[u;alpha;q;de;dt];
F_long=@(X)longDeriv(X,theta,v,w, beta,da,dr,p,r, Cthr, phi);
Coeff_long = findJacobian(F_long,i0_long);

VT=Mach*a;
dt=thetaEngine;

%Lateral motion

i0_lat=[beta;p;r;da;dt];
F_lat=@(X)latDeriv(X,alpha,de,q, theta,phi,u, v, w, Cthr);
Coeff_lat=findJacobian(F_lat, i0_lat);

beta = 0/180*pi; % [rad]

%calculating dim values
longdimvalues=longDvar(Coeff_long, alpha);
latedimvalues=lateDvar(Coeff_lat);

%finding prime #nertias
Jxx_Prime=(Jxx*Jzz-Jxz^2)/(Jzz);
Jzz_Prime=(Jxz*Jzz-Jxz^2)/(Jxx);
Jxz_Prime=Jxz/(Jxx*Jzz-Jxz^2);

%calculating longitudinal matrice

```

```

A_lon = [(longdimvalues(1,1)/m) (longdimvalues(1,2)/m)
(longdimvalues(1,3)/(m-w)) (-g*cos(theta));
(longdimvalues(2,1)/m) (longdimvalues(2,2)/m)
(longdimvalues(2,3)/(m+u)) (-g*sin(theta));
(longdimvalues(3,1)/Jyy) (longdimvalues(3,2)/Jyy)
(longdimvalues(3,3)/Jyy) 0;
0 0 0
0]

E_lon = [1 (longdimvalues(1,6)/m) 0 0;
0 (1-longdimvalues(2,6)/m) 0 0;
0 (-longdimvalues(3,6)/Jyy) 1 0;
0 0 0 1]

%calculating lateral matrice

A_lat = [(latedimvalues(1,1)/m) (latedimvalues(1,3)/(m+w)) ...
(latedimvalues(1,4)/(m-u)) (g*cos(theta)) 0;
((latedimvalues(2,1)/Jxx_Prime) +
(latedimvalues(3,1)*Jxz_Prime)) ...
((latedimvalues(2,3)/Jxx_Prime) +
(latedimvalues(3,3)*Jxz_Prime)) ...
((latedimvalues(2,4)/Jxx_Prime) + (latedimvalues(3,4)*Jxx_Prime))
0 0;
((latedimvalues(2,1)*Jxz_Prime) + (latedimvalues(3,1)/
Jzz_Prime)) ...
((latedimvalues(2,3)*Jxz_Prime) + (latedimvalues(3,3)/
Jzz_Prime)) ...
((latedimvalues(2,4)*Jxz_Prime) + (latedimvalues(3,4)/Jzz_Prime))
0 0;
0 1 tan(theta) 0 0;
0 0 acos(theta) 0 0]

eigen_longitude=eig(inv(E_lon)*A_lon)
eigen_latitude=eig(A_lat)

fprintf("-----Comments on the part E-----\n");
fprintf("Since it can be observed from the eigenvalues for\n");
fprintf("longitudinal motion,\n");
fprintf("they are not significantly larger than zero. In this case, it\n");
fprintf("is possible to say that\n");
fprintf("the longitudinal motion of A/C is dynamically stable.\n\n");
fprintf("Since it can be observed from the eigen values for lateral\n");
fprintf("motion, \n");
fprintf("they have a significant larger real part than zero.\n");
fprintf("Therefore, in this case, it is possible \n");
fprintf("to say that the lateral motion of A/C isn't dynamically\n");
fprintf("stable.\n");

%%%%%%%%%%%%PLOTTING%%%%%%%%%%%%
%
```

%FOR PART C)

```
figure(1)
subplot(3,1,1)
plot(t,mach_exp);
xlabel('Time(s)');
ylabel('Mach number');
grid on
title('Mach number vs time')
ylim([-1 1]);
```

```
subplot(3,1,2)
plot(t,alpha_exp);
xlabel('Time(s)');
ylabel('Alpha(rad)');
grid on
title('alpha vs time')
ylim([-1 1]);
```

```
subplot(3,1,3)
plot(t,beta_exp);
xlabel('Time(s)');
ylabel('Beta(rad)');
grid on
title('beta vs time')
ylim([-1 1]);
```

```
figure(2)
```

```
subplot(3,1,1)
plot(t,p_exp);
xlabel('Time(s)');
ylabel('p(rad/s)');
grid on
title('p vs time')
ylim([-1 1]);
```

```
subplot(3,1,2)
plot(t,q_exp);
xlabel('Time(s)');
ylabel('q(rad/s)');
grid on
title('q vs time')
ylim([-1 1]);
```

```
subplot(3,1,3)
plot(t,r_exp);
xlabel('Time(s)');
ylabel('r(rad/s)');
grid on
title('r vs time')
ylim([-1 1]);
```

```
figure(3)

subplot(3,1,1)
plot(t,phi_exp);
xlabel('Time(s)');
ylabel('phi(rad)');
grid on
title('phi vs time')
ylim([-1 1]);

subplot(3,1,2)
plot(t,theta_exp);
xlabel('Time(s)');
ylabel('theta(rad)');
grid on
title('theta vs time')
ylim([-1 1]);

subplot(3,1,3)
plot(t,psi_exp);
xlabel('Time(s)');
ylabel('psi(rad)');
grid on
title('psi vs time')
ylim([-1 1]);

%FOR PART D)

%%For longitudinal

figure(4)
subplot(3,1,1)
plot(t,mach_exp2);
xlabel('Time(s)');
ylabel('Mach number');
grid on
title('Mach number vs time')

subplot(3,1,2)
plot(t,alpha_exp2);
xlabel('Time(s)');
ylabel('Alpha(rad)');
grid on
title('alpha vs time')

subplot(3,1,3)
plot(t,beta_exp2);
xlabel('Time(s)');
ylabel('Beta(rad)');
grid on
title('beta vs time')

figure(5)
```

```
subplot(3,1,1)
plot(t,p_exp2);
xlabel('Time(s)');
ylabel('p(rad/s)');
grid on
title('p vs time')

subplot(3,1,2)
plot(t,q_exp2);
xlabel('Time(s)');
ylabel('q(rad/s)');
grid on
title('q vs time')

subplot(3,1,3)
plot(t,r_exp2);
xlabel('Time(s)');
ylabel('r(rad/s)');
grid on
title('r vs time')

figure(6)

subplot(3,1,1)
plot(t,phi_exp2);
xlabel('Time(s)');
ylabel('phi(rad)');
grid on
title('phi vs time')

subplot(3,1,2)
plot(t,theta_exp2);
xlabel('Time(s)');
ylabel('theta(rad)');
grid on
title('theta vs time')

subplot(3,1,3)
plot(t,psi_exp2);
xlabel('Time(s)');
ylabel('psi(rad)');
grid on
title('psi vs time')

%%For lateral

figure(7)
subplot(3,1,1)
plot(t,mach_exp3);
xlabel('Time(s)');
ylabel('Mach number');
grid on
```

```
title('Mach number vs time')
```

```
subplot(3,1,2)
plot(t,alpha_exp3);
xlabel('Time(s)');
ylabel('Alpha(rad)');
grid on
title('alpha vs time')
```

```
subplot(3,1,3)
plot(t,beta_exp3);
xlabel('Time(s)');
ylabel('Beta(rad)');
grid on
title('beta vs time')
```

```
figure(8)
```

```
subplot(3,1,1)
plot(t,p_exp3);
xlabel('Time(s)');
ylabel('p(rad/s)');
grid on
title('p vs time')
```

```
subplot(3,1,2)
plot(t,q_exp3);
xlabel('Time(s)');
ylabel('q(rad/s)');
grid on
title('q vs time')
```

```
subplot(3,1,3)
plot(t,r_exp3);
xlabel('Time(s)');
ylabel('r(rad/s)');
grid on
title('r vs time')
```

```
figure(9)
```

```
subplot(3,1,1)
plot(t,phi_exp3);
xlabel('Time(s)');
ylabel('phi(rad)');
grid on
title('phi vs time')
```

```
subplot(3,1,2)
plot(t,theta_exp3);
xlabel('Time(s)');
ylabel('theta(rad)');
grid on
```

```

title('theta vs time')

subplot(3,1,3)
plot(t,psi_exp3);
xlabel('Time(s)');
ylabel('psi(rad)');
grid on
title('psi vs time')

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%SUBFUNCTIONS%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%

function coefficients=calcoef(alpha,da,de,dr,p,q,r,VT,beta)
global cbar b THETA1 THETA2 THETA3...
    THETA4 THETA5 THETA6 THETA7 THETA8 THETA9 THETA10 THETA11
    THETA12...
    THETA13 THETA14 THETA15 THETA16 THETA17 THETA18 THETA19 THETA20
    THETA21...
    THETA22 THETA23 THETA24 THETA25 THETA26 THETA27 THETA28 THETA29
    THETA30...
    THETA31 THETA32 THETA33 THETA34 THETA35 THETA36 THETA37 THETA38
    THETA39 THETA40...
    THETA41 THETA42 THETA43 THETA44 THETA45
%
-----
% A/C: aerodynamic force and moment coefficients
%
-----
CD = THETA1 + THETA2*alpha + THETA3*alpha*q*cbar/2/VT +
    THETA4*alpha*de + ...
    THETA5*alpha^2 + THETA6*alpha^2*q*cbar/2/VT + THETA7*alpha^2*de +
    THETA8*alpha^3 +...
    THETA9*alpha^3*q*cbar/2/VT + THETA10*alpha^4;
CY = THETA11*beta + THETA12 *p*b/2/VT + THETA13*r*b/2/VT + THETA14*da
    + THETA15*dr;
CL = THETA16 + THETA17*alpha + THETA18*q*cbar/2/VT + THETA19*de +...
    THETA20*alpha*q*cbar/2/VT + THETA21*alpha^2 + THETA22*alpha^3 +
    THETA23*alpha^4;
%
-----
Cl = THETA24*beta + THETA25 *p*b/2/VT + THETA26*r*b/2/VT + THETA27*da
    + THETA28*dr;
Cm = THETA29 + THETA30*alpha + THETA31*q*cbar/2/VT + THETA32*de +...
    THETA33*alpha*q*cbar/2/VT + THETA34*alpha^2*q*cbar/2/VT +
    THETA35*alpha^2*de +...
    THETA36*alpha^3*q*cbar/2/VT + THETA37*alpha^3*de + THETA38*alpha^4;
Cn = THETA39*beta + THETA40 *p*b/2/VT + THETA41*r*b/2/VT + THETA42*da
    + THETA43*dr +...
    THETA44*beta^2 + THETA45*beta^3;
%
-----
coefficients=[CD,CY,CL,Cl,Cm,Cn];
end

```

```

function
    [total_force,total_moment,weight]=calcFM(coefficients,alpha,Cthr,theta,phi,VT)

global m Sref b thetaEngine beta g ro cbar

%Aero Coefficients
CD=coefficients(1);
CY=coefficients(2);
CL=coefficients(3);
Cl=coefficients(4);
Cm=coefficients(5);
Cn=coefficients(6);

%Thrust force calculation

Qd=0.5*ro*VT^2;
C_bT=R_2(thetaEngine);
TL=Qd*Sref*Cthr;
thrust=C_bT*[TL;0;0];

%Construction of force vector
D=Qd*Sref*CD;
Y=Qd*Sref*CY;
L=Qd*Sref*CL;

%transformation from wind frame to body frame
F_b=R_2(-alpha)*R_3(beta)*[-D;Y;-L];

%construction of gravity vector
Gx=-m*g*sin(theta);
Gy=m*g*cos(theta)*sin(phi);
Gz=m*g*cos(theta)*cos(phi);

weight=transpose([Gx Gy Gz]);

%Construction of moment vector
Mx=Qd*Sref*Cl*b;
My=Qd*Sref*Cm*cbar;
Mz=Qd*Sref*Cn*b;

total_moment=transpose([Mx My Mz]);

%total forces acting on body frame
Xb=F_b(1)+thrust(1);
Yb=F_b(2)+thrust(2);
Zb=F_b(3)+thrust(3);

total_force=transpose([Xb Yb Zb]);

end

function
    [total_force,total_moment,weight]=calcFM_2(coefficients,alpha,Cthr,theta,phi,VT,t

```

```

global m Sref b beta g ro cbar

%Aero Coefficients
CD=coefficients(1);
CY=coefficients(2);
CL=coefficients(3);
Cl=coefficients(4);
Cm=coefficients(5);
Cn=coefficients(6);

%Thrust force calculation

Qd=0.5*ro*VT^2;
C_bT=R_2(thetaengine);
TL=Qd*Sref*Cthr;
thrust=C_bT*[TL;0;0];

%Contruction of force vector
D=Qd*Sref*CD;
Y=Qd*Sref*CY;
L=Qd*Sref*CL;

%transformation from wind frame to body frame
F_b=R_2(-alpha)*R_3(beta)*[-D;Y;-L];

%construction of gravity vector
Gx=-m*g*sin(theta);
Gy=m*g*cos(theta)*sin(phi);
Gz=m*g*cos(theta)*cos(phi);

weight=transpose([Gx Gy Gz]);

%Construction of moment vector
Mx=Qd*Sref*Cl*b;
My=Qd*Sref*Cm*cbar;
Mz=Qd*Sref*Cn*b;

total_moment=transpose([Mx My Mz]);

%total forces acting on body frame
Xb=F_b(1)+thrust(1);
Yb=F_b(2)+thrust(2);
Zb=F_b(3)+thrust(3);

total_force=transpose([Xb Yb Zb]);

end

function eqns=wing_level(u)

global gamma da de dr p q r phi VT beta

```

```

%introducing the parameters

alpha=u(1);
de=u(2);
Cthr=u(3);

%redefining alpha for ODEfcn
theta=alpha+gamma;

coefficients=calcoef(alpha,da,de,dr,p,q,r,VT,beta);
[total_force,total_moment,weight]=calcFM(coefficients,alpha,
    Cthr,theta,phi,VT);

%6 DOF ==> 6 Eqns
eqns(1,1)=total_force(1)+weight(1);
eqns(2,1)=total_force(2)+weight(2);
eqns(3,1)=total_force(3)+weight(3);
eqns(4,1)=total_moment(1);
eqns(5,1)=total_moment(2);
eqns(6,1)=total_moment(3);

end

function R1=R_1(q)
R1=[1 0 0;0 cos(q) -sin(q);0 sin(q) cos(q)];
end
function R2=R_2(q)
R2=[cos(q) 0 sin(q);0 1 0;-sin(q) 0 cos(q)];
end
function R3=R_3(q)
R3=[cos(q) -sin(q) 0;sin(q) cos(q) 0;0 0 1];
end

function D=odefcn(t,Z,alpha,de,Cthr)
    %Changing parameters
    u=Z(1);
    v=Z(2);
    w=Z(3);
    p=Z(4);
    q=Z(5);
    r=Z(6);
    phi=Z(7);
    theta=Z(8);
    psi=Z(9);

    %constants needed from global defined constants

    global m Jxx Jyy Jzz Jxz da dr VT beta
    VT=sqrt(u^2+v^2+w^2);
    coefficients=calcoef(alpha,da,de,dr,p,q,r,VT,beta);
    [total_force,total_moment,weight]=calcFM(coefficients,alpha,
        Cthr,theta,phi,VT);

    %Translational equation of motion

```

```

u_dot=total_force(1)/m+weight(1)/m+r*v-q*w;
v_dot=total_force(2)/m+weight(2)/m+p*w-r*v;
w_dot=total_force(3)/m+weight(3)/m+q*u-p*v;

%Rotational equation of motion

dummy_A=total_moment(1)+p*q*Jxz-r*q*(Jzz-Jyy);
dummy_B=total_moment(3)-p*q*(Jyy-Jxx)-q*r*Jxz;

p_dot=(dummy_A*Jzz+dummy_B*Jxz)/(Jxx*Jzz-Jxz^2);
q_dot=(total_moment(2)-(p*r)*(Jxx-Jzz)-Jxz*(p^2-r^2))/(Jyy);
r_dot=(dummy_B+p_dot*Jxz)/(Jzz);

phi_dot=p+q*sin(phi)*tan(theta)+r*cos(phi)*tan(theta);
theta_dot=q*cos(phi)-r*sin(phi);
psi_dot=q*sin(phi)/cos(theta)+r*cos(phi)/cos(theta);

%returning the desired parameters

D(1,1)=u_dot;
D(2,1)=v_dot;
D(3,1)=w_dot;

D(4,1)=p_dot;
D(5,1)=q_dot;
D(6,1)=r_dot;

D(7,1)=phi_dot;
D(8,1)=theta_dot;
D(9,1)=psi_dot;

end

function D=odefcn2(t,time_step,Z,alpha,de,Cthr,dr,da)
u=Z(1);
v=Z(2);
w=Z(3);

p=Z(4);
q=Z(5);
r=Z(6);

phi=Z(7);
theta=Z(8);
psi=Z(9);

Cthr=Cthr(round(t/time_step)+1);
de=de(round(t/time_step)+1);
dr=dr(round(t/time_step)+1);
da=da(round(t/time_step)+1);

global m Jxx Jyy Jzz Jxz VT beta

```

```

VT=sqrt(u^2+v^2+w^2);

coefficients=calcoef(alpha,da,de,dr,p,q,r,VT,beta);
[total_force,total_moment,weight]=calcFM(coefficients,alpha,
    Cthr,theta,phi,VT);

%Translational equation of motion

u_dot=total_force(1)/m+weight(1)/m+r*v-q*w;
v_dot=total_force(2)/m+weight(2)/m+p*w-r*v;
w_dot=total_force(3)/m+weight(3)/m+q*u-p*v;

%Rotational equation of motion

dummy_A=total_moment(1)+p*q*Jxz-r*q*(Jzz-Jyy);
dummy_B=total_moment(3)-p*q*(Jyy-Jxx)-q*r*Jxz;

p_dot=(dummy_A*Jzz+dummy_B*Jxz)/(Jxx*Jzz-Jxz^2);
q_dot=(total_moment(2)-(p*r)*(Jxx-Jzz)-Jxz*(p^2-r^2))/(Jyy);
r_dot=(dummy_B+p_dot*Jxz)/(Jzz);

phi_dot=p+q*sin(phi)*tan(theta)+r*cos(phi)*tan(theta);
theta_dot=q*cos(phi)-r*sin(phi);
psi_dot=q*sin(phi)/cos(theta)+r*cos(phi)/cos(theta);

%returning the desired parameters

D(1,1)=u_dot;
D(2,1)=v_dot;
D(3,1)=w_dot;

D(4,1)=p_dot;
D(5,1)=q_dot;
D(6,1)=r_dot;

D(7,1)=phi_dot;
D(8,1)=theta_dot;
D(9,1)=psi_dot;

end

function longval=longDvar(C,alpha)
global m Jyy Sref cbar ro h VT

C_xu      = C(1,1);
C_xAlpha  = C(1,2);
C_xq      = C(1,3);
C_xde     = C(1,4);
C_xdt     = C(1,5);
C_zu      = C(2,1);
C_zAlpha  = C(2,2);
C_zq      = C(2,3);
C_zde     = C(2,4);
C_zdt     = C(2,5);

```

```

C_mu      = C(3,1);
C_mAlpha  = C(3,2);
C_mq      = C(3,3);
C_mde     = C(3,4);
C_mdt     = C(3,5);

X_u       = (C_xu*ro*VT*Sref)/(2*m);
X_w       = (C_xAlpha*ro*VT*Sref)/(2*m);
X_q       = (C_xq*ro*VT*Sref*cbar)/(4*m);
X_de      = (C_xde*ro*VT^2*Sref)/(2*m);
X_dt      = (C_xdt*ro*VT^2*Sref)/(2*m);
X_wdot    = (ro*pi*cbar*Sref*sin(alpha))/4;
Z_u       = (C_zu*ro*VT*Sref)/(2*m);
Z_w       = (C_zAlpha*ro*VT*Sref)/(2*m);
Z_q       = (C_zq*ro*VT*Sref*cbar)/(4*m);
Z_de      = (C_zde*ro*VT^2*Sref)/(2*m);
Z_dt      = (C_zdt*ro*VT^2*Sref)/(2*m);
Z_wdot    = (-ro*pi*cbar*Sref*cos(alpha))/4;
M_u       = (C_mu*ro*VT*Sref*cbar)/(2*Jyy);
M_w       = (C_mAlpha*ro*VT*Sref*cbar)/(2*Jyy);
M_q       = (C_mq*ro*VT*Sref*cbar^2)/(4*Jyy);
M_de      = (C_mde*ro*VT^2*Sref*cbar)/(2*Jyy);
M_dt      = (C_mdt*ro*VT^2*Sref*cbar)/(2*Jyy);
M_wdot    = (ro*pi*cbar*Sref*(h-cbar/2))/4;

%returning the desired parameters

longval = [X_u X_w X_q X_de X_dt X_wdot; Z_u Z_w Z_q Z_de Z_dt Z_wdot;
           M_u M_w M_q M_de M_dt M_wdot];
end

function latval=lateDvar(X)
global m Jxx Jzz Jxz Sref ro VT b

Jxx_Prime=(Jxx*Jzz-Jxz^2)/(Jzz);
Jzz_Prime=(Jxz*Jzz-Jxz^2)/(Jxx);

C_yBeta=X(1,1);
C_yp=X(1,2);
C_yr=X(1,3);
C_yda=X(1,4);
C_ydr=X(1,5);
C_lBeta=X(2,1);
C_lp=X(2,2);
C_lr=X(2,3);
C_lda=X(2,4);
C_ldr=X(2,5);
C_nBeta=X(3,1);
C_np=X(3,2);
C_nr=X(3,3);
C_nda=X(3,4);
C_ndr=X(3,5);

Y_v=(C_yBeta*ro*VT*Sref)/(2*m);

```

```

Y_Beta=(C_yBeta*ro*VT^2*Sref)/(2*m);
Y_p=(C_yp*ro*VT*Sref*b)/(4*m);
Y_r=(C_yr*ro*VT*Sref*b)/(4*m);
Y_da=(C_yda*ro*VT^2*Sref)/(2*m);
Y_dr=(C_ydr*ro*VT^2*Sref)/(2*m);
L_v=(C_lBeta*ro*VT*Sref*b)/(2*Jxx_Prime);
L_Beta=(C_lBeta*ro*VT^2*Sref*b)/(2*Jxx_Prime);
L_p=(C_lp*ro*VT*Sref*b^2)/(4*Jxx_Prime);
L_r=(C_lr*ro*VT*Sref*b^2)/(4*Jxx_Prime);
L_da=(C_lda*ro*VT^2*Sref*b)/(2*Jxx_Prime);
L_dr=(C_ldr*ro*VT^2*Sref*b)/(2*Jxx_Prime);
N_v=(C_nBeta*ro*VT*Sref*b)/(2*Jzz_Prime);
N_Beta=(C_nBeta*ro*VT^2*Sref*b)/(2*Jzz_Prime);
N_p=(C_np*ro*VT*Sref*b^2)/(4*Jzz_Prime);
N_r=(C_nr*ro*VT*Sref*b^2)/(4*Jzz_Prime);
N_da=(C_nda*ro*VT^2*Sref*b)/(2*Jzz_Prime);
N_dr=(C_ndr*ro*VT^2*Sref*b)/(2*Jzz_Prime);

%returning the desired parameters

latval = [Y_v Y_Beta Y_p Y_r Y_da Y_dr;L_v L_Beta L_p L_r L_da L_dr;
          N_v N_Beta N_p N_r N_da N_dr];
end

function F=longDeriv(X,theta,v,w, beta,da,dr,p,r, Cthr, phi)

global Sref ro

%Variables to find
u =X(1);
alpha=X(2);
q=X(3);
de=X(4);
dt=X(5);

VT = sqrt(u^2+v^2+w^2);

coefficients=calcoef(alpha,da,de,dr,p,q,r,VT,beta);
[total_force,total_moment,weight]=calcFM_2(coefficients,alpha,
Cthr,theta,phi,VT,dt);

%returning the desired parameters

% From stability derivatives equations
F(1)=(2*total_force(1,1))/(ro*VT^2*Sref); %X
F(2)=(2*total_force(3,1))/(ro*VT^2*Sref); %Z
F(3)=coefficients(5); %M

end

function F=latDeriv(X,alpha,de,q, theta,phi,u, v, w, Cthr)

global Sref ro VT beta

```

```

beta=X(1);
p=X(2);
r=X(3);
da=X(4);
dr=X(5);

coefficients=calcoef(alpha,da,de,dr,p,q,r,VT,beta);
[total_force,total_moment,weight]=calcFM(coefficients,alpha,
    Cthr,theta,phi,VT);

%returning the desired parameters

F(1)=(total_force(2)*2)/(ro*VT^2*Sref);
F(2)=coefficients(4);
F(3)=coefficients(6);

end

function jac = findJacobian(F,X)
delta_X = 10^-6;
jac = zeros(length(F(X)),length(X));
for i=1:length(X)
    delta_Vec=zeros(length(X),1);
    delta_Vec(i)=delta_X;
    delta_F=F(X+delta_Vec)-F(X-delta_Vec);
    jac(:,i)=delta_F(:)/delta_X/2;
end
end

Warning: Trust-region-dogleg algorithm of FSOLVE cannot handle non-
square
systems; using Levenberg-Marquardt algorithm instead.
-----Comments on the part A-----
MATLAB optimization toolbox is used to find alpha_trim,de_trim and
    Cthr_trim
by considering trim conditions.

alpha_trim =

    0.5245

de_trim =

   -0.3140

Cthr_trim =

    0.8943

-----Comments on the part B-----
dCm/dAlpha = -0.485643
Since Longitudinal Static Stability Derivative is negative,

```

therefore, the A/C is stable in longitudinal direction plane.

$dCl/d\beta = -0.071000$

$dCn/d\beta = 0.234000$

Since lateral Static Stability Derivative is negative and directional static stability is positive. In this case we can conclude that The A/C is stable in directional plane.

-----Comments on the part C-----

From the plots that we have, It can be observed that all of the variables stayed

almost constant and during the simulation time period p, q, r was stayed at zero. As a result, we can conclude that this means the A/C is in wings level trimmed steady flight.

-----Comments on the part D-----

A/C is stable in longitudinal direction since the longitudinal stability derivative contains no positive value.

A/C is unstable in the lateral direction since $dCn/d\beta$ does not have negative value and $dCl/d\beta$ is negative.

$A_{lon} =$

0	0.0000	0.0000	-8.4883
0.0000	-0.0000	-0.0000	-4.9105
0	-0.0000	-0.0000	0
0	0	0	0

$E_{lon} =$

1.0000	0.0049	0	0
0	1.0085	0	0
0	-0.1190	1.0000	0
0	0	0	1.0000

$A_{lat} =$

$1.0e+03 *$

-0.0000	-0.0000	0.0000	0.0085	0
-0.0000	-0.0000	-1.9938	0	0
0.0000	0.0000	-0.0000	0	0
0	0.0010	0.0006	0	0
0	0	0.0010	0	0

$eigen_{longitude} =$

$1.0e-04 *$

0.0000
-0.3278
-0.0011
0

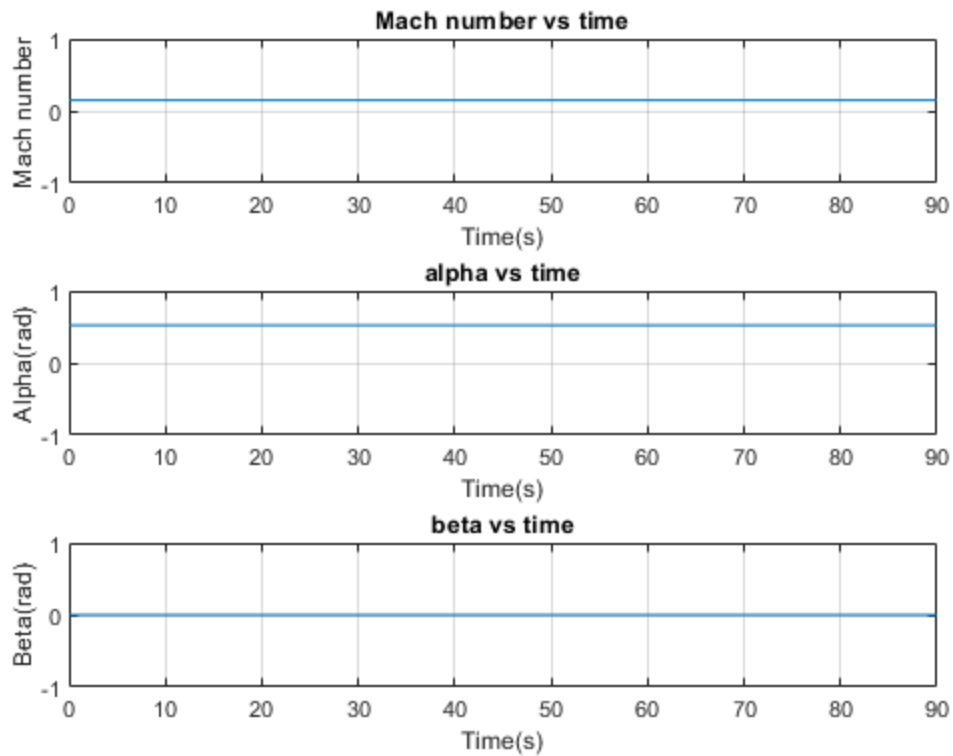
`eigen_latitude =`

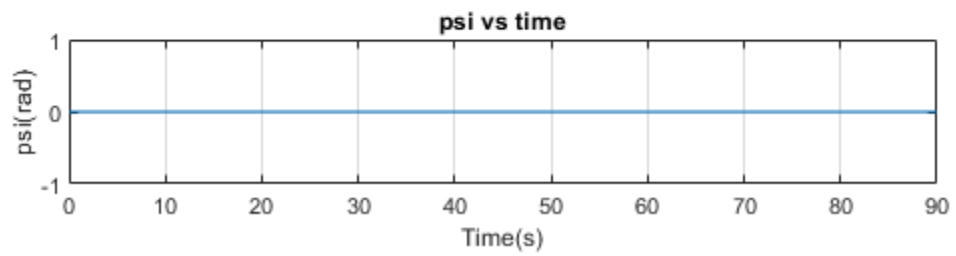
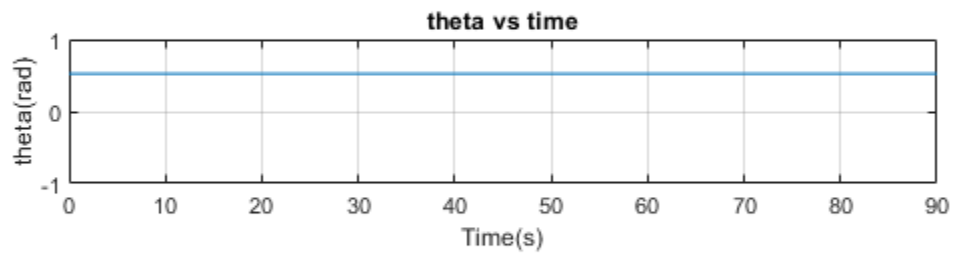
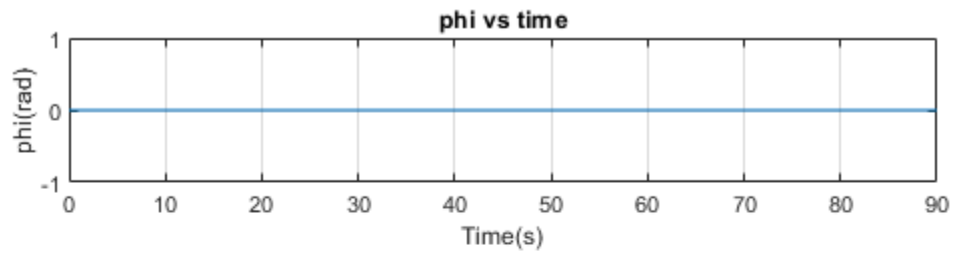
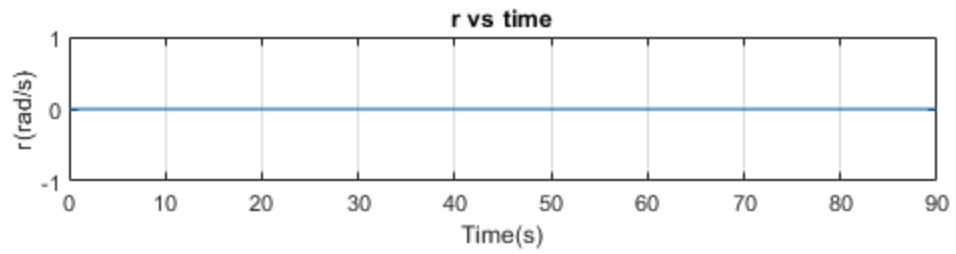
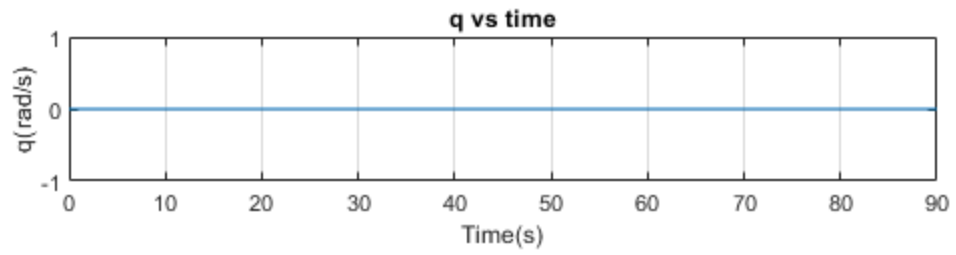
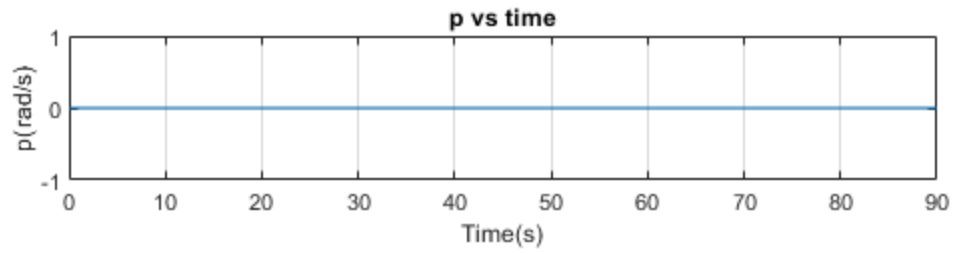
`0.0000 + 0.0000i`
`-0.5651 + 0.5671i`
`-0.5651 - 0.5671i`
`0.5651 + 0.5670i`
`0.5651 - 0.5670i`

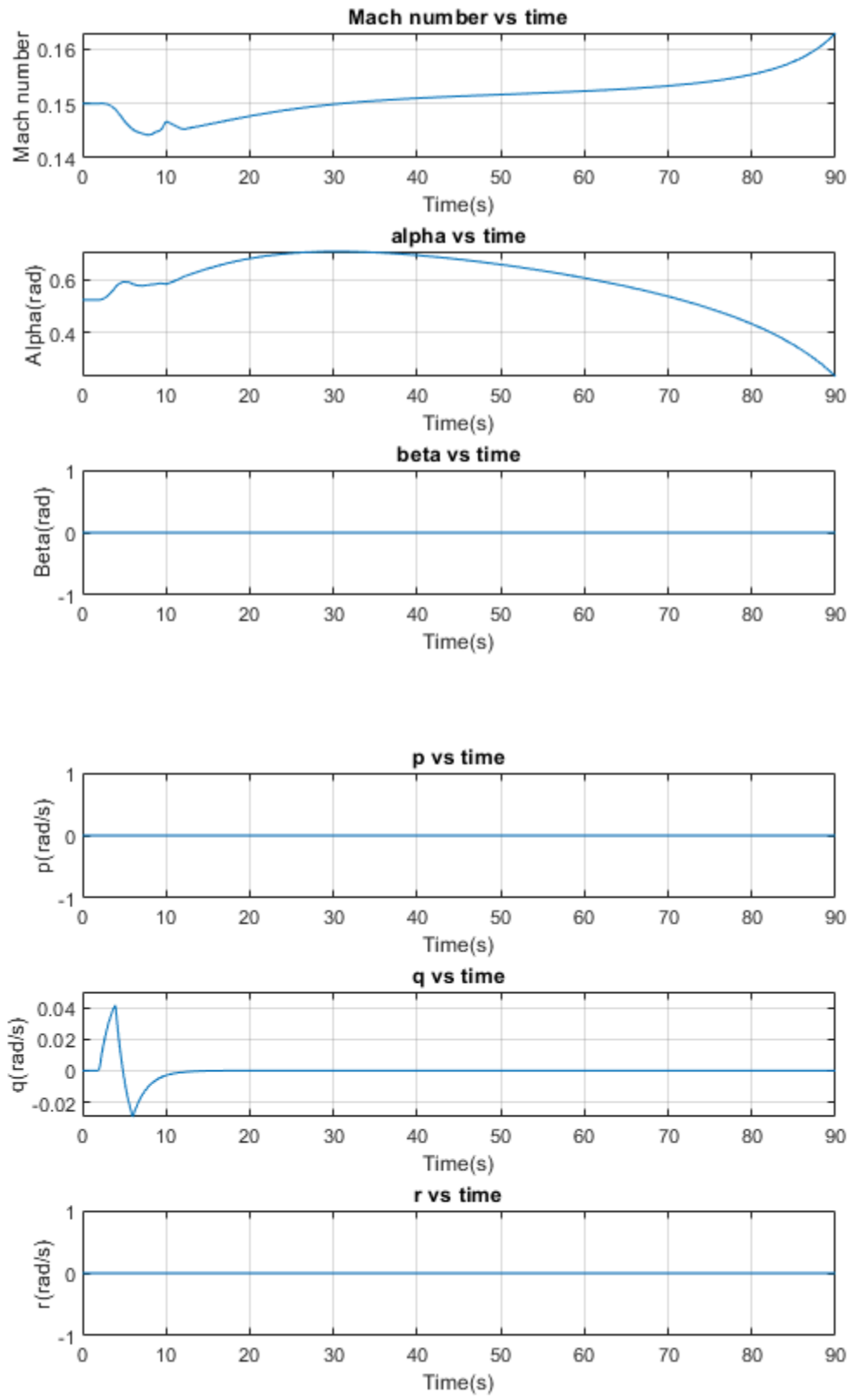
-----Comments on the part E-----

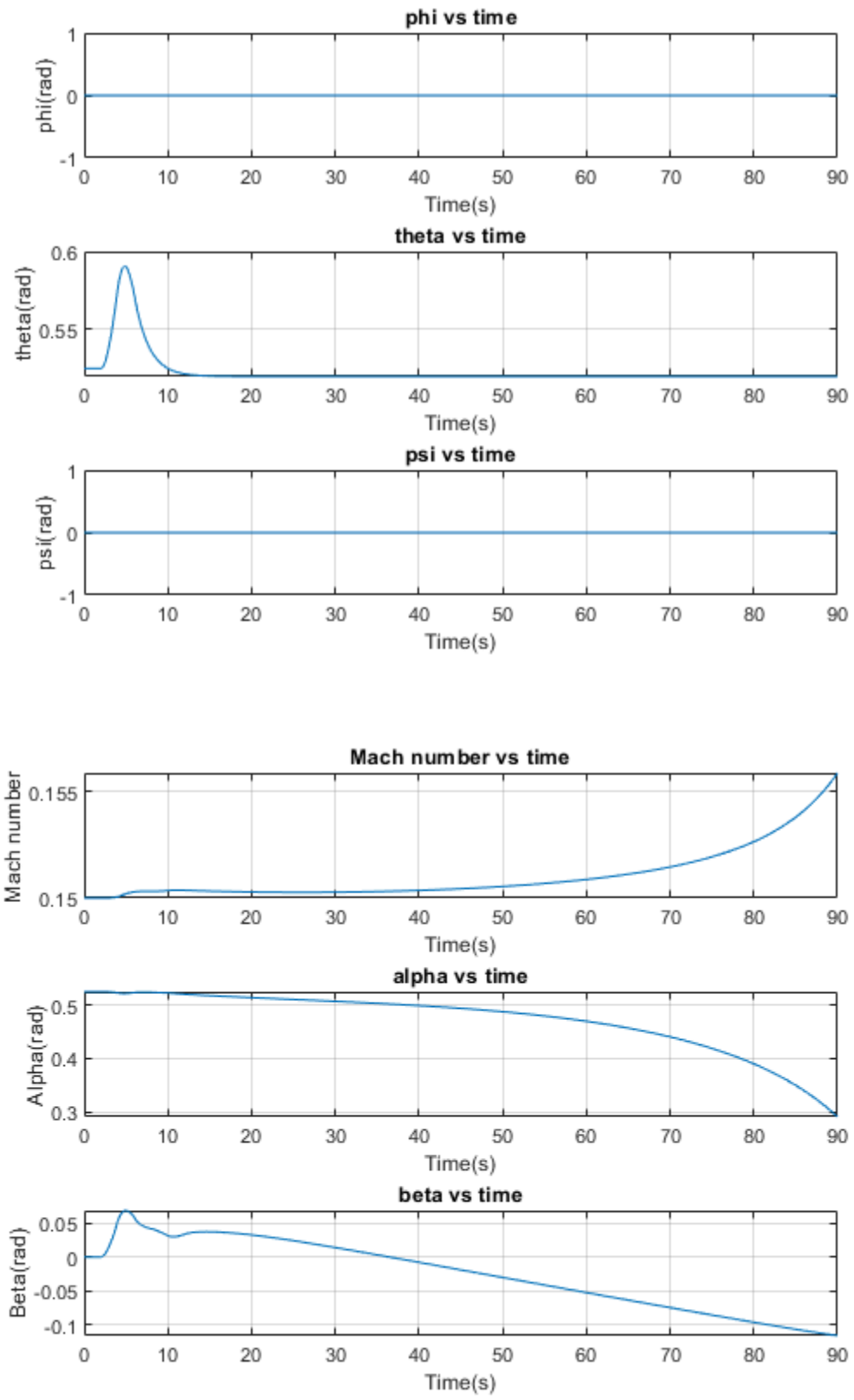
Since it can be observed from the eigenvalues for longitudinal motion, they are not significantly larger than zero. In this case, it is possible to say that the longitudinal motion of A/C is dynamically stable.

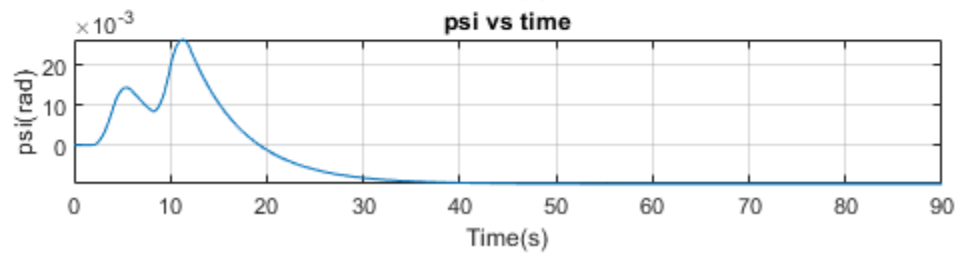
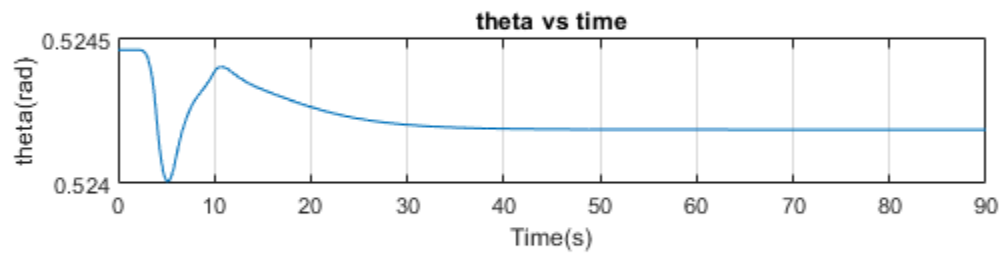
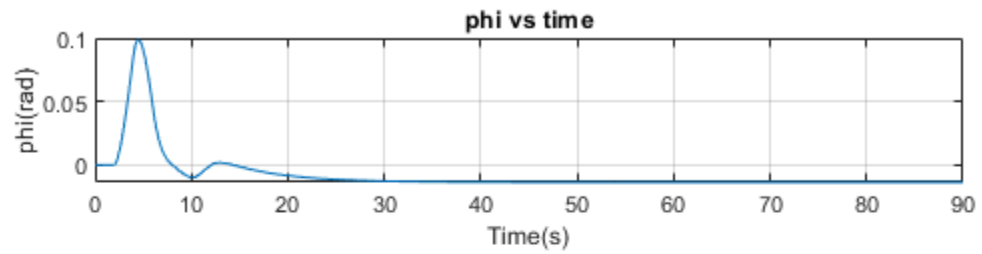
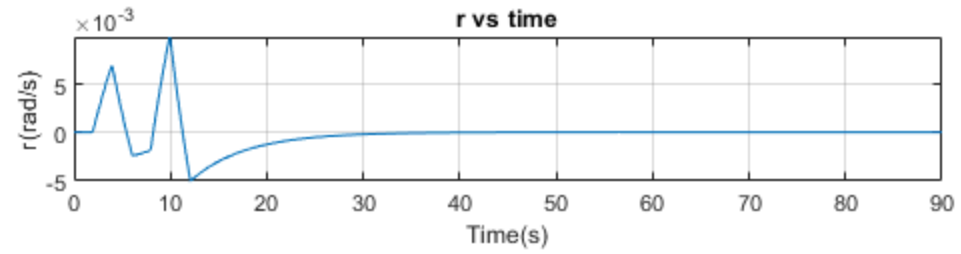
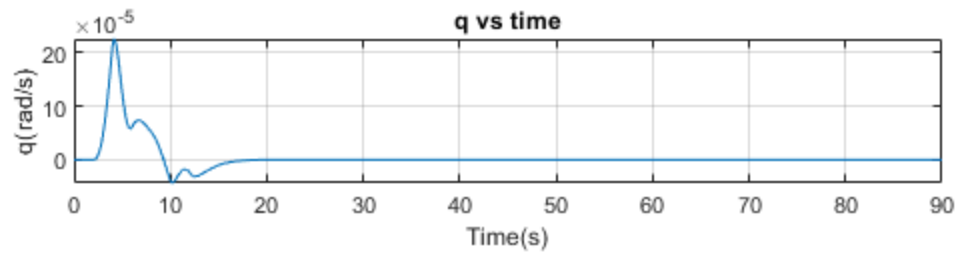
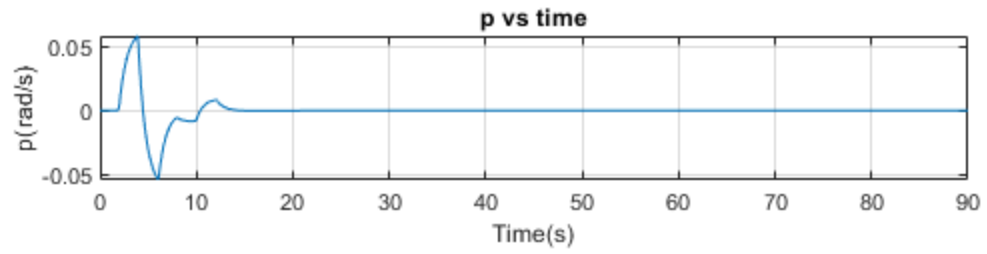
Since it can be observed from the eigen values for lateral motion, they have a significantly larger real part than zero. Therefore, in this case, it is possible to say that the lateral motion of A/C isn't dynamically stable.











Published with MATLAB® R2020b